

CRYPTOGRAPHY & SECURITY

COMPREHENSIVE QUICK SILVER REFERENCE CHEAT SHEET

TOPICS COVERED

- Digital Signatures
- SSH in CTF Environments
- Symmetric vs Asymmetric
- PKI & Digital Certificates
- PGP & GPG Commands
- Quantum Computing Impact
- SSH Authentication & Key Mgmt
- AES Symmetric Encryption
- Post-Quantum Cryptography (PQC)

SECTION 1 — DIGITAL SIGNATURES

1.1 What Is a Digital Signature?

A **Digital Signature** is a cryptographic mechanism used to prove:

- **Authenticity** → Who created/signed the data?
- **Integrity** → Has the data been modified?
- **Non-repudiation** → The signer cannot easily deny having signed it.

IMPORTANT

A digital signature does **NOT** provide confidentiality. If you want confidentiality, you need encryption.

1.2 How Digital Signatures Work

Step 1 — Hashing: The original document is passed through a cryptographic hash function (e.g., SHA-256) producing a fixed-size digest.

Document → SHA-256 → Hash / Digest

Step 2 — Signing: The signer uses their **Private Key** to create a digital signature over the hash/data.

Document → SHA-256 → Hash → Private Key → Digital Signature

Step 3 — Verification: The receiver verifies the signature using the signer's **Public Key** and compares the computed hash.

Received Document → SHA-256 → Computed Hash
Signature + Public Key → Verification Success → Valid Signature & Integrity Confirmed

1.3 Digital Signature vs Encryption

FEATURE	DIGITAL SIGNATURE	ENCRYPTION
Main Purpose	Authentication & Integrity	Confidentiality
Uses Private Key	Yes	Depends on scheme
Uses Public Key	Yes	Yes (in asymmetric encryption)
Prevents Modification Detection	Yes	Not necessarily by itself
Provides Confidentiality	✗ No	✓ Yes
Proves Origin	✓ Yes	✗ Not by itself

REMEMBER

Signature: "Who signed this, and was it changed?"

Encryption: "Can anyone else read this?"

SECTION 2 & 3 — PKI & DIGITAL CERTIFICATES

2.1 What Is PKI?

PKI (Public Key Infrastructure) is a system of Public/Private keys, Digital Certificates, Certificate Authorities (CAs), Trust relationships, and Certificate validation mechanisms. Its purpose is to establish trust between identities and public keys.

2.2 Why Do We Need Certificates?

When connecting to `https://example.com`, your browser must verify if the public key actually belongs to example.com.

Identity + Public Key → Digital Certificate (Signed by CA)

3.1 & 3.2 Digital Certificates & HTTPS

A digital certificate is a digitally signed document binding an identity to a public key. It contains: Domain name, Public key, CA info, Validity period, Serial number, Signature algorithm, and Extensions.

When visiting HTTPS sites, the browser checks: Validity, Expiration, Domain match, Trusted CA signature, Chain validity, and Revocation status.

SECTION 4 & 5 — CHAIN OF TRUST & LET'S ENCRYPT

Root CA (Trust Anchor) → Intermediate CA → Leaf / Server Certificate (example.com)

- **Root CA:** Top of the hierarchy; embedded in OS/browsers as Trust Anchors.
- **Intermediate CA:** Certified by Root CA; issues leaf certificates to protect Root CA keys.
- **Leaf / Server Certificate:** Presented by the end server (e.g., `tryhackme.com`).

LET'S ENCRYPT

A free, automated, and widely supported Certificate Authority (CA) designed to make HTTPS deployment seamless.

SECTION 6 - 10 — SSH AUTHENTICATION & MANAGEMENT

SSH (Secure Shell) provides secure remote access over TCP Port 22.

SSH Password vs Key Authentication

- **Password Auth:** Uses traditional Username + Password.
- **Key Auth:** Uses Asymmetric Key Pairs (Client keeps Private Key, Server stores Public Key).

SSH PASSPHRASE & PROTECTION

Private SSH keys can be encrypted locally with a passphrase. The passphrase unlocks the private key locally and is **never** sent to the SSH server.

SSH Directory & File Permissions

Stored in `~/.ssh/`. Correct permissions are critical for SSH to function securely:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
chmod 600 id_rsa
```

FILE	DESCRIPTION	RECOMMENDED PERMISSION
id_rsa	RSA Private Key (Keep secret!)	chmod 600 (-rw-----)
id_rsa.pub	RSA Public Key (Shareable)	chmod 644 (-rw-r--r--)
authorized_keys	Public keys allowed to authenticate as user	chmod 600 (-rw-----)
known_hosts	Previously seen server host keys	chmod 644 (-rw-r--r--)

SECTION 11 - 14 — SSH COMMANDS & CTF WORKFLOWS

Key Generation & Connection

```
# Generate 4096-bit RSA Key Pair
ssh-keygen -t rsa -b 4096

# Custom Filename
ssh-keygen -t rsa -b 4096 -f ctf_key

# Connect with Private Key
chmod 600 id_rsa
ssh -i id_rsa user@192.168.1.10
```

SSH Private Key Passphrase Recovery (CTF)

Encrypted SSH Key → ssh2john → hash.txt → John the Ripper → Passphrase

```
ssh2john id_rsa > hash.txt
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
john --show hash.txt
```

SECTION 15 - 19 — PGP & GPG CHEAT SHEET

PGP (Pretty Good Privacy) is an ecosystem for encryption, signatures, and email security. **GPG (GNU Privacy Guard)** is the open-source implementation of the OpenPGP standard.

Essential GPG Commands

```
# Import a Key
gpg --import key.asc

# List Public / Secret Keys
gpg --list-keys
gpg --list-secret-keys

# Decrypt a File
gpg -d encrypted_file.gpg
gpg --output decrypted.txt --decrypt encrypted_file.asc
```

GPG Passphrase Cracking (CTF)

```
gpg2john private_key.asc > gpg_hash.txt
john --wordlist=/usr/share/wordlists/rockyou.txt gpg_hash.txt
```

SECTION 20 - 24 — AES & SYMMETRIC ENCRYPTION

AES (Advanced Encryption Standard) is a symmetric block cipher operating on fixed **128-bit blocks**. Same secret key is used for both encryption and decryption.

Algorithm	Key Size	Block Size	Security Status
DES	56 bits	64 bits	❌ Obsolete / Insecure
AES-128	128 bits	128 bits	🟢 Secure (Classical) / 🟡 Reduced (Quantum)
AES-192	192 bits	128 bits	🟢 Secure
AES-256	256 bits	128 bits	🟢 High Security / Quantum Resistant

Authenticated Encryption

AES alone doesn't provide integrity. Modern applications use modes like **AES-GCM** to deliver both confidentiality and integrity/authentication.

Symmetric vs Asymmetric Summary

Feature	Symmetric Encryption	Asymmetric Encryption
Keys	One shared secret key	Public + Private key pair
Speed	Very Fast	Slower / Computationally Heavy
Large Data	Excellent (Bulk Encryption)	Inefficient (Small payload/keys)
Examples	AES, DES, 3DES	RSA, ECC, Diffie-Hellman
Primary Use	Disk, File, VPN, Bulk TLS	Signatures, Key Exchange, Auth

SECTION 25 - 28 — QUANTUM COMPUTING IMPACT

Shor's Algorithm vs Grover's Algorithm

- **Shor's Algorithm:** Solves integer factorization and discrete logarithms efficiently.
CRITICAL THREAT Completely breaks **RSA, ECC, and Diffie-Hellman**.
- **Grover's Algorithm:** Provides a quadratic speedup for searching unstructured data.
MODERATE IMPACT Halves the effective security bits of symmetric ciphers.
 - AES-128 → ≈ 64-bit security (Vulnerable)
 - AES-256 → ≈ 128-bit security (Safe margin)

Cryptography Category	Examples	Quantum Threat / Impact
Public-Key Encryption	RSA, ECC	Major Threat (Shor's Algorithm)
Key Exchange	Diffie-Hellman	Major Threat (Shor's Algorithm)
Symmetric Cipher	AES-128	Reduced Security (64-bit margin)
Symmetric Cipher	AES-256	Strong Security Margin (128-bit margin)
Hash Functions	SHA-256	Safe (Reduced search margin, still secure)

SECTION 29 - 33 — POST-QUANTUM CRYPTOGRAPHY (PQC)

PQC algorithms are designed to resist both classical and quantum computer attacks without requiring quantum hardware.

NIST PQC Standards

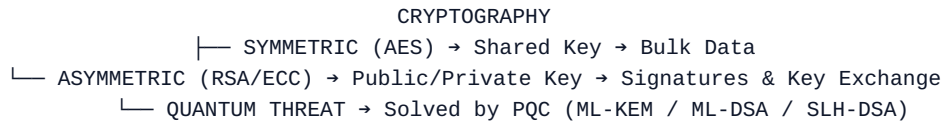
- **ML-KEM (Module-Lattice Key Encapsulation):** Used for Key Encapsulation (Lattice-based).
- **ML-DSA (Module-Lattice Digital Signature):** Used for Digital Signatures (Lattice-based).
- **SLH-DSA (Stateless Hash-Based Digital Signature):** Signature standard based on hash functions.

CRYPTO AGILITY & HNDL

Crypto Agility: Designing systems so algorithms can be swapped seamlessly without full architecture redesigns.

Harvest Now, Decrypt Later (HNDL): Threat actors capturing encrypted traffic today to decrypt once quantum computers become available. This mandates immediate PQC planning.

SECTION 34 & 35 — MAP & KEY DEFINITIONS



SECTION 36 & 37 — EXAM QUICK REVIEW & CHEAT SHEET

```
# SSH Quick Reference
ssh-keygen -t rsa -b 4096 -f ctf_key
chmod 600 id_rsa && chmod 700 ~/.ssh
ssh -i id_rsa user@target_ip

# Passphrase Cracking
ssh2john id_rsa > hash.txt && john --wordlist=rockyou.txt hash.txt
pgp2john key.asc > hash.txt && john --wordlist=rockyou.txt hash.txt

# GPG Operations
gpg --import key.asc
gpg --list-keys
gpg --output file.txt --decrypt file.gpg
```

SECTION 38 — GOLDEN RULES TO MEMORIZE

Rule 1: Hashing ≠ Encryption

Hashing is 1-way. Encryption is 2-way (reversible with key).

Rule 2: Signature ≠ Encryption

Signatures prove authenticity/integrity. Encryption provides confidentiality.

Rule 3: Private vs Public Key

Private Key = Keep Secret! Public Key = Share Freely.

Rule 4: AES = Symmetric

Same secret key for encryption and decryption.

Rule 5: RSA/ECC = Asymmetric

Uses key pairs for signatures and key establishment.

Rule 6: Certificate

Certificate = Identity + Public Key + CA Signature.

Rule 7: PKI

Trust management ecosystem for keys, certificates, and CAs.

Rule 8: Shor's Threat

Shor's Algorithm completely breaks RSA, ECC, and DH.

Rule 9: Grover's Threat

Grover's Algorithm halves symmetric key security (Use AES-256).

Rule 10: PQC Goal

Quantum-resistant algorithms (ML-KEM, ML-DSA) to counter quantum threats.

FINAL REVISION SUMMARY TABLE

TOPIC	KEY CONCEPT TO REMEMBER
Hash	Integrity / Fixed-size fingerprint
Digital Signature	Sign with Private Key, verify with Public Key (Authenticity + Integrity)
Certificate & PKI	Identity ↔ Public Key binding managed by CA trust hierarchy
SSH Permissions	<code>chmod 600</code> for private keys & authorized_keys, <code>700</code> for <code>~/.ssh</code>
PGP / GPG	OpenPGP standard ecosystem for file/email encryption and signing
AES	Symmetric block cipher (128-bit blocks). Preferred: AES-256
Shor's Algorithm	Quantum algorithm breaking RSA, ECC, and Diffie-Hellman
Grover's Algorithm	Quantum search speedup reducing AES key security by half
PQC Standards	ML-KEM (Key encapsulation), ML-DSA & SLH-DSA (Digital signatures)
Crypto Agility / HNDL	Ability to swap algorithms / Shielding against Harvest-Now-Decrypt-Later